

1 reference

```
private void logout_button_Click(object sender, EventArgs e)
{
    var logoutConfirmationDialog = new LogoutConfirmationDialog();

    logoutConfirmationDialog.ConfirmLogOut += OnLogoutConfirmed;

    logoutConfirmationDialog.Location = new Point(
        (Width - logoutConfirmationDialog.Width) / 2,
        (Height - logoutConfirmationDialog.Height) / 2
    );

    Controls.Add(logoutConfirmationDialog);
    logoutConfirmationDialog.BringToFront();
}
```

1 reference

```
private async void logout_confirm_button_Click(object sender, EventArgs e)
{
    await networkService.LogoutAsync();
    ConfirmLogOut?.Invoke();
}
```

1 reference

```
public async Task LogoutAsync()
{
    StopLobbyChatReceiveLoop();

    if (!IsConnected)
    {
        return;
    }

    await SendLogoutPacket();
    await ReceiveLogoutResultPacket();
}
```

```

public async Task SendLogoutPacket()
{
    var stream = tcpClient.GetStream();
    byte[] packet = packetService.CreateLogoutRequestPacket();
    await stream.WriteAsync(packet, 0, packet.Length);
}

```

```

case Opcodes.C2S_Logout:
    if (peer.State != ClientState.Connected)
    {
        await HandleLogoutAsync(peer, netStream);
    }
    break;

```

```

async Task HandleLogoutAsync(ClientPeer peer, NetworkStream network)
{
    RemovePeerFromLobby(peer);
    peers.TryRemove(peer.UserId, out _);

    peer.UserId = 0;
    peer.Username = string.Empty;
    peer.State = ClientState.Connected;

    byte[] packet = packetService.CreateLogoutResultPacket(true);
    await network.WriteAsync(packet, 0, packet.Length);
}

```

```

public async Task<bool> ReceiveLogoutResultPacket()
{
    PacketData packet = await ReadPacketAsync();
    return packet.Payload.Length > 0 && packet.Payload[0] == 1;
}

```

```
private async void logout_confirm_button_Click(object sender, EventArgs e)
{
    await networkService.LogoutAsync(); //finished
    ConfirmLogOut?.Invoke(); //this got called
}
```

Reference

```
private void ShowHomeFromSession()
{
    if (string.IsNullOrEmpty(CurrentUserSession))
    {
        ShowMain();
        return;
    }

    var homeControl = new HomeControl(CurrentUserSession);
    homeControl.LogoutRequested += Logout; //here
    homeControl.GoToSetting += ShowSetting;
    homeControl.PlayOnline += ShowFindLobby;
    ShowScreen(homeControl);
}
```

```
private void OnLogoutConfirmed()
{
    LogoutRequested?.Invoke();
}
```

Reference

```
private void Logout()
{
    CurrentUserSession = null;
    ShowMain();
}
```